Quiz 16-3

- Why does Kruskal's greedy choice never force a heavier edge later?
- What structure detects 'forms a cycle', and why is it (not) the bottleneck?

힌트 보기

개념 정리 · 안전한 간선과 사이클 탐지

- **안전한 간선**: 현재까지 선택한 간선 집합을 A라 하자. Kruskal은 현재 A에 추가해도 사이클을 만들지 않는 모든 간선 중 가중치가 최소인 간선 $e = (u, v)$를 선택한다.
  S를 u가 속한 A의 연결요소라고 하면, e는 컷 $(S, VsetminusS)$을 가로지른다. 이 컷을 가로지르는 모든 간선은 서로 다른 연결요소를 잇기 때문에 선택 후보에 속하며, 그 가중치는 $w(e)$ 이상이다. 따라서 e는 현재 선택된 간선들을 가로지르지 않는 컷의 최소 간선이다.
  A가 어떤 MST에 포함된다는 귀납 가정 아래, cut property에 의해 $Acupe$를 포함하는 MST도 존재한다. 즉, Kruskal의 선택은 안전하다.
- **Union-Find(Disjoint Set)**: 현재 채택한 간선의 연결요소를 관리한다. 두 끝점의 대표 원소가 같으면 이미 두 정점 사이에 경로가 있으므로 간선을 추가할 때 사이클이 생긴다.
  경로 압축(path compression)과 랭크 기준 합치기(union by rank)를 함께 사용하면, 연속된 Find/Union 연산의 분할상환 시간은 연산당 $O(alpha(n))$이다. 여기서 n은 정점 수이고, α는 역 아커만 함수이다.

강의 슬라이드 Ch16 p29 Quiz

정답 보기

풀이

① **막다른 길이 없는 이유.** 현재까지 채택한 간선 집합을 A, 다음으로 선택하는 간선을 $e = (u, v)$라 하자. u가 속한 A의 연결요소를 S라 하면, e는 컷 $(S, VsetminusS)$의 최소 가중치 간선이다. 이 컷을 가로지르는 모든 간선이 현재 선택 가능한 간선이기 때문이다. 이 컷을 가로지르는 A의 간선은 없으므로 cut property에 의해 e는 안전하다.

> 교환 논증 — 1. A를 포함하는 MST T를 택한다. $einT$이면 이미 $Acupe$가 MST에 포함되므로 증명이 끝난다.
> 2. $eotinT$이면 T에 e를 추가했을 때 사이클 하나가 생긴다. 이 사이클에서 컷 $(S, VsetminusS)$을 가로지르는 간선 f를 선택한다.
> 3. 이 컷을 가로지르는 간선은 A에 속하지 않으므로 $fotinA$이다. 또한 e가 컷의 최소 간선이므로 $w(e)lew(f)$이다.
> 4. 따라서 $T' = T - f + e$는 $Acupe$를 포함하는 스패닝 트리이고,
> $w(T') = w(T) - w(f) + w(e)lew(T)$.
> 5. T가 MST이므로 T'도 MST이다. 따라서 Kruskal의 현재 선택은 최적해를 유지하며, 이후 더 비싼 최종 해를 강제하지 않는다.

② **사이클 탐지.** Union-Find로 두 끝점이 같은 집합인지 검사한다.

> 같은 집합 = 이미 경로가 있음 = 간선 추가 시 사이클.

③ **병목이 아닌 이유.** $n = |V|, e = |E|$라 하자.
사이클 탐지: Union-Find의 Find 연산으로 두 끝점의 연결요소를 확인하고, 간선을 채택하면 Union으로 합친다.
Union-Find 비용: 경로 압축과 랭크 기준 합치기를 사용하면 전체 연산 비용은 $O(ealpha(n))$ (분할상환)이고, 초기화 비용은 $O(n)$이다.
정렬 비용: 간선을 가중치 순으로 정렬하는 데 $O(eloge)$가 든다.
따라서 연결 그래프에서 일반적인 비교 정렬을 사용하는 Kruskal의 전체 시간 복잡도는 $O(eloge)$이며, 주된 병목은 간선 정렬이지 Union-Find가 아니다.

최종 답

cut property 덕분에 각 선택이 안전. 사이클 탐지는 Union-Find(거의 상수), 병목은 간선 정렬 $O(e \log e)$

한 줄 정리 · Kruskal의 시간은 **정렬**이 지배한다. Union-Find는 거의 공짜.